ФСП | САМАРСКИЙ ПОЛИТЕХ Опорный университет

// кейсовое задание хакатона

Платформа алгоритмических соревнований

Веб-платформа для обычных контестов и командных соревнований с проверкой честности — чтобы результатам можно было доверять.

ФОРМАТЫ

Обычный контест
Дуэль 1×1
Командные соревнования

ДОКУМЕНТ

Постановка задачи и критерии оценки

ОЦЕНКА

100 баллов
7 критериев

// 01

Постановка задачи

Кратко: нужно сделать рабочий прототип веб-платформы для соревнований по программированию. На ней можно проводить обычные контесты или командные соревнования, а организатор может проверить, что результаты заработаны честно.

Главная ценность: соревноваться честно и интересно. Участник получает удобный формат, в том числе соревнование с другим человеком один на один или в команде. Организатор получает инструменты, чтобы убедиться, что результаты заработаны честно.

Чем продукт не является

Это не полноценный аналог Codeforces с архивом задач и рейтингом за многие годы. Это не LMS (система для онлайн-обучения) и не прокторинговая система общего назначения. В центре продукта — соревнование и его прозрачность.

// 02

Полное описание задачи

ЦЕЛЬ

Чтобы провести соревнование по программированию, в том числе в командном формате, и доверять его результатам, должно хватать одной платформы.

ФОРМА

Веб-приложение с пользовательским интерфейсом, проверкой решений, хранением данных и интерфейсом организатора.

РЕЗУЛЬТАТ

Работающий прототип, который можно проверить, и обоснованная концепция продукта.

ПРОБЛЕМА

Соревнования один на один и в паре проводят на самодельных инструментах. Онлайн-контесты трудно контролировать: организатор не видит, как участник пришёл к решению.

// 03

Требования к решению

Обязательные требования

- две роли: **участник** и **организатор**. Регистрацию и вход можно имитировать;
- задачи с условием, ограничениями и тестами, автоматическая проверка решений в изолированной среде;
- обычный контест с таблицей результатов;
- два парных режима: **дуэль 1 на 1** и **команда из двух человек**;
- решение для **проверки честности** соревнования, доступное организатору.

Реализация, дополнительный функционал и режимы соревнований — **на усмотрение команды**.

Объясните, какую проблему решает каждое улучшение и почему оно нужно вашей аудитории. Важнее польза и связность с основным сценарием, чем количество фич. Дополнительные функции **не должны заменять** работающий основной сценарий.

Задачи и проверка решений

Задача состоит из условия, ограничений по времени и памяти и набора тестов. Решение принимается хотя бы на одном языке программирования. Вердикты: как минимум **OK / WA / TLE / RE / CE**.

Код участника выполняется в **изолированной среде с лимитами**. Это обязательно: запуск кода без изоляции — серьёзная ошибка.

Обычный контест

Организатор создаёт контест: выбирает задачи, время начала и длительность. Участники регистрируются, решают задачи и видят таблицу результатов. Правила подсчёта (**ICPC, IOI или свои**) команда выбирает сама и обосновывает выбор.

Командные соревнования

Оба режима должны работать в одном прототипе.

РЕЖИМ 1

Дуэль 1 на 1

Два участника одновременно решают одни и те же задачи. Побеждает тот, кто быстрее или решил больше.

РЕЖИМ 2

Командные соревнования

Команда решает общий набор задач вместе. Как распределить работу — по задачам, по очереди или иначе — решает команда-разработчик.

В обоих режимах нужны **подбор соперника или напарника** (хотя бы по ссылке-приглашению) и **отображение хода соревнования в реальном времени**.

Записи и проверки

Команда должна **придумать решение для проверки** честности соревнования. Как это будет устроено и как реализовано — команда решает сама.

// 04

Базовые сценарии проверки

Жюри проверяет прототип **по этим сценариям**. Необходимо отразить это в скринкасте.

СЦЕНАРИЙ 01

Создание контеста

Организатор создаёт задачу с тестами и контест с этой задачей. Участник находит контест и регистрируется.

СЦЕНАРИЙ 02

Решение и вердикт

Участник отправляет правильное и неправильное решения. Он получает разные вердикты, таблица результатов обновляется.

СЦЕНАРИЙ 03

Дуэль

Участник приглашает соперника по ссылке. Оба решают в реальном времени и видят прогресс друг друга. В конце определяется победитель.

СЦЕНАРИЙ 04

Командный режим

Двое участников объединяются в команду и решают общий набор задач. Результат засчитывается команде.

СЦЕНАРИЙ 05

Проверка честности

После соревнования организатор проверяет прохождение участника с помощью решения, которое придумала команда.

СЦЕНАРИЙ 06

Небезопасный код

Решение с бесконечным циклом или попыткой обратиться к файловой системе. Платформа выдаёт вердикт и продолжает работать.

// 05

Технические требования

ОБЩИЕ

Веб-приложение. Выбор стека не ограничен. Можно использовать готовые библиотеки и сервисы, в том числе для проверки решений.

ДЕМОНСТРАЦИОННЫЕ ДАННЫЕ

Несколько задач с тестами, контест, демо-аккаунты участников и организатора, пример данных для проверки честности. Жюри проходит сценарии, ничего не подготавливая вручную.

Что сдать

- репозиторий с кодом и **ссылку на работающий прототип**;
- **README.md**: описание продукта, стек, порядок запуска;
- демо-доступы для ролей;
- описание того, что имитируется, и известных ограничений;
- презентацию.

// 06

Критерии оценки

01	**Работоспособность MVP** Обязательный минимум и сценарии проверки работают от начала до конца	25
02	**Дополнительные режимы** Фичи сверх MVP, которые команда придумала сама: польза, оригинальность, качество реализации	20
03	**Командный режим** Оба режима (дуэль и команда из двух): механика, реальное время, удобство	10
04	**Записи и проверки** Польза решения для проверки честности, удобство для организатора, приватность участников	15
05	**UX/UI** Понятно ли участнику и организатору, хватает ли обратной связи	10
06	**Защита проекта** Ясная демонстрация, честный рассказ об ограничениях, ответы на вопросы	10
07	**Техническая экспертиза** Архитектура, безопасность проверки решений, масштабируемость, качество кода	10
	Итого максимальная сумма баллов	100

ШТРАФ запуск кода участников без изоляции — **до −10 баллов**

// 07

Презентация решения

РЕКОМЕНДУЕМАЯ СТРУКТУРА

- для кого продукт и какую проблему он решает;
- концепция и механика командных режимов;
- **чем вы отличаетесь** от других решений — отдельным слайдом;
- **демонстрация сценариев проверки** вживую или записью экрана, встроенной в презентацию;
- как устроена проверка честности;
- архитектура, стек, что имитируется, ограничения;
- куда развивать продукт.

На защите **отделяйте то, что работает, от макетов и планов**.

// 08

Вопросы для размышления

? **Как не дать участнику обойти проверку честности, и где граница с приватностью?**

? **Что делать, если в дуэли у одного из участников оборвалось соединение?**

? **Как быстро проверять решения, если одновременно идёт 100 дуэлей?**

? **Какие данные действительно помогают организатору проверить честность, а какие только мешают?**

? **Как определить победителя дуэли, если оба решили одинаково?**

? **Нужно ли организатору видеть вклад каждого участника в командном режиме?**